Graph Structure Learning for Traffic Prediction

Mahmood Amintoosi[1*]

[1*]Division of Computer Science, Department of Applied Mathematics, Faculty of Mathematical Sciences, Ferdowsi University of Mashhad, Bahonar Av., Mashhad, 91775, Khorasan-e Razavi, IRAN, ORCID:0000-0001-9640-6475.

Corresponding author(s). E-mail(s): m.amintoosi@um.ac.ir;

Abstract

Accurate traffic forecasting remains challenging due to complex spatial and temporal dependencies. Existing graph-based methods typically rely on predefined adjacency matrices constructed from physical proximity, although physical connections may not adequately represent functional dependencies in traffic data. To address this limitation, we investigate a Graph Structure Learning (GSL) framework in which the adjacency matrix is learned directly from traffic observations rather than predefined from physical connections. We consider both contemporaneous and multi-lag learned graph structures and evaluate them with spatial and spatiotemporal forecasting models on the real-world SZ-Taxi and Los-loop datasets. The results show that learned graph structures can improve traffic forecasting, with the largest and most consistent gains obtained when lag-specific structures are consumed according to their temporal alignment. On Los-loop, the best learned multi-lag configuration reduces RMSE by 14.5% relative to the graph-free baseline at the first forecasting horizon and by up to 43.0% relative to the physical-graph baseline across the evaluated horizons. On SZ-Taxi, the learned multi-lag structure provides only limited gains, reflecting the much sparser structure recovered from the data. Additional experiments with matched-sparsity graphs show that the observed gains on Los-loop cannot be explained by sparsity alone. Overall, the results indicate that learning the graph structure from traffic data can provide useful predictive information, although its effectiveness depends on the dataset and on how the learned structure is incorporated into the forecasting model. The learned graphs are interpreted as statistical dependencies rather than causal relationships. The paper code is available at https://github.com/mamintoosi/TGCN-GSL-PyTorch.

Keywords: Traffic Forecasting, Graph Knowledge Discovery, Graph Structure Learning, Graph Convolutional Networks, T-GCN, Multi-Lag Graph Learning

1 Introduction

Recent advances in graph knowledge discovery have opened new possibilities for understanding complex urban systems, particularly in transportation networks. This paper applies the principles of graph knowledge discovery to traffic prediction, and examines whether learning the graph structure from traffic data improves forecasting relative to graphs that are predefined from physical connectivity. In recent years, traffic prediction has garnered significant attention in smart urban transportation systems. In navigation, ride-hailing, and route planning, accurate traffic prediction plays a crucial role in reducing congestion and increasing the efficiency of transportation networks. Traffic prediction involves analyzing traffic conditions, including flow and speed, extracting traffic patterns, and forecasting traffic trends on urban roads. Due to the complex spatial and temporal dependencies involved, this problem remains challenging and is an active area of research.

The spatial dependencies in this problem arise from the fact that traffic conditions on one road segment are statistically related to conditions on other segments through the topology of the urban road network and through shared demand patterns. Traffic volume also changes dynamically over time, affected by short-term and long-term patterns, which underscores the temporal dependencies in traffic data.

These relationships motivate sophisticated models that can capture joint spatio-temporal structure. A critical limitation of many existing approaches, however, is their reliance on physical proximity or predefined road-network topology to construct the adjacency matrix. Physical connectivity need not coincide with functional co-movement in the data: distant roads can exhibit strong traffic correlations because of commuting patterns, while adjacent roads can have weak statistical dependence. Moreover, dense proximity graphs can degrade graph-convolutional models through oversmoothing, so that a graph-based forecaster can be worse than the same backbone without a graph. These observations raise a natural question: if the adjacency matrix is a modeling choice rather than a fixed physical fact, should it be estimated from the traffic data themselves?

Multiple traffic prediction methods, including neural networks, statistical approaches, and Bayesian methods, model temporal evolution but neglect spatial dependencies [1]. To describe spatial features, many studies use graph-based methods [2–5]. In [6], a Temporal Graph Convolutional Network (T-GCN) was proposed for traffic prediction. T-GCN combines a Graph Convolutional Network (GCN) with a Gated Recurrent Unit (GRU): the GCN captures spatial dependence and the GRU captures temporal dependence. The combined model achieves strong results on standard traffic benchmarks.

Existing graph-based methods typically build the adjacency matrix from road proximity or distance [6]. While such graphs capture geographic connectivity, they need not represent the statistical dependency structure present in the observed speeds. Attention-based methods [7–9] reweight relationships between nodes over time, but they usually operate on an initial graph that is still defined by heuristics or physical proximity; they change how a given structure is used more than which structure is the right one.

Our work studies *graph structure learning* (GSL) for traffic forecasting: the adjacency matrix is inferred from the traffic series rather than taken as predefined. Continuous optimization methods for directed acyclic graphs, such as NOTEARS [10] and DAGMA [11], make such estimation practical. Relative to implicit reweighting over a fixed topology, GSL produces an explicit binary (or weighted) adjacency that can be inspected as a statistical dependency graph over sensors. We interpret that graph as descriptive statistical structure under the estimator's assumptions, not as a causal map of traffic influence.

This paper evaluates that idea under a controlled experimental design on two public benchmarks, SZ-Taxi and Los-loop. In addition to the physical road-network graph, we include a *graph-free* identity control, so that any contribution of learned structure can be measured against both a standard baseline and a model that uses no spatial graph. We consider a single contemporaneous learned graph (and its binary symmetrization, cGSL) as well as *multi-lag* learned graphs in which lag-specific structures are estimated from an explicitly lag-stacked input. Because a learned adjacency can be applied as one static operator or aligned with input timesteps in a recurrent backbone, we further examine how the *consumption* of multi-lag graphs affects forecasting accuracy. Finally, matched-sparsity controls address the methodological concern that any gain might be due only to the sparsity of the learned graph rather than to the placement of its edges.

The experimental results confirm that learning graph structure from traffic data can improve forecasting, but they also qualify the claim. Relative to the physical graph, learned and even identity adjacencies often reduce error. Relative to the graph-free control, the largest and most consistent gains on Los-loop appear when lag-specific structures are consumed in a temporally aligned manner. On SZ-Taxi, where the recovered multi-lag structure is extremely sparse, learned variants remain close to the graph-free baseline. Thus, the usefulness of a learned graph depends on the dataset and on how the learned structure is incorporated into the forecasting model; it is not a universal replacement for either the physical graph or a graph-free model.

The contributions of this paper are the following:

1. We investigate GSL for traffic forecasting with GCN and T-GCN backbones, replacing a predefined physical adjacency with an adjacency estimated from the traffic data.
2. We compare learned graphs not only to the physical baseline but also to a graph-free control, and we evaluate both a contemporaneous construction and a multi-lag construction of the learned structure.
3. We study how multi-lag graphs are consumed by the forecasting model (static union versus lag-aligned recurrent use, including a gated mixing variant), which separates the estimated structure from the mechanism that uses it.
4. We report five-seed experiments on SZ-Taxi and Los-loop, together with matched-sparsity controls, showing that the benefit of learned structure is dataset-dependent and cannot be attributed to sparsity alone on the dataset where gains are clearest.

In the following sections, we review background on traffic prediction and GCN/T-GCN models (Section 2). We then present the GSL framework used to estimate the adjacency matrix and its integration with GCN and T-GCN (Section 3). Experimental settings are described in Section 4, results in Section 5, and discussion in Section 6. Limitations are stated in Section 7, and conclusions in Section 8.

2 Background and Related Work

The goal of traffic forecasting is to predict future traffic conditions from historical data. Traffic conditions can include metrics such as speed, flow, or density; in our experiments we focus on traffic speed without loss of generality.

We model the road network as a graph $G = (V, E)$, where $V = \{v_1, \ldots, v_N\}$ represents N road segments (sensors) as nodes, E represents edges between connected roads, and $\mathbf{A} \in \{0, 1\}^{N \times N}$ is the adjacency matrix encoding those connections (or, in learned variants, estimated statistical adjacency). The traffic data are represented by a feature matrix $X \in \mathbb{R}^{N \times P}$, where P is the length of the historical series and $X_t \in \mathbb{R}^N$ is the vector of speeds at time t. The forecasting task is to learn a mapping

$$[X_{t+1}, \ldots, X_{t+T}] = f\big(G; [X_{t-n}, \ldots, X_t]\big), \tag{2.1}$$

where n is the number of historical steps used as input and T is the prediction horizon.

2.1 Overview of Traffic Prediction Approaches

Traffic prediction methods can be broadly categorized as follows.

Classical prediction methods include ARIMA models [12, 13], support vector regression [14], seasonal ARIMA variants [15], and Kalman filtering approaches [16]. These methods are effective for basic temporal patterns on individual series but often struggle with network-scale spatial dependence and strong nonlinearity.

Classical deep learning methods include feedforward networks [17], spatio-temporal residual networks [18], interactive temporal recurrent convolutional networks [19], and recurrent variants such as GRU and LSTM [20, 21]. They capture rich temporal structure and some spatial coupling, but their Euclidean assumptions limit the modeling of irregular network topology.

Graph-based and temporal graph methods explicitly model network topology. Representative models include GCNs [22, 23], T-GCN [6], attention-based temporal GCNs (A3T-GCN) [7], dynamic spatial–temporal attention networks [8], and a range of recent spatio-temporal graph convolutional, recurrent, and transformer architectures [9, 24–28]. These methods model topology and time jointly, but they typically rely on a predefined graph that may not match the dependency structure in the data.

Comprehensive reviews are available elsewhere [26, 29, 30]. Our focus is the role of the adjacency matrix itself. A bibliometric analysis of 784 traffic-forecasting articles (2000–2025) confirms the rise of graph-based methods and the continued dominance of heuristic graph construction (Appendix A).

Since this paper builds on GCN and T-GCN, we next summarize those two backbones concisely; fuller treatments appear in [6, 22].

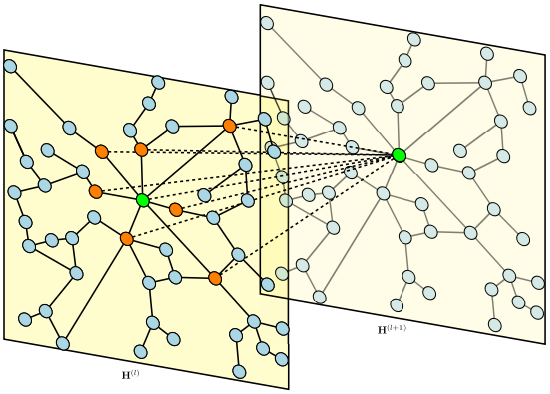


Fig. 1: A graph convolutional architecture with layers l and $(l+1)$ computing $\mathbf{H}^{(l+1)}$ as in (2.2). Neighbors are defined by the adjacency matrix $\mathbf{A}$. In the standard setting $\mathbf{A}$ comes from the physical road network; in this work adjacency is also estimated from traffic data.

2.2 Graph Convolutional Network

Graph Convolutional Networks extend convolution to graph-structured data [22, 31] and have been applied widely, including community detection [32, 33], semi-supervised learning [22, 34], and recommendation [35]. Figure 1 illustrates a graph-convolutional layer used for traffic prediction: the green node is the target sensor and orange nodes are its graph neighbors. Traditional pipelines take $\mathbf{A}$ from the physical road network; our proposed pipeline instead estimates node adjacencies from traffic data.

The layer-wise propagation rule is [22]

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right), \tag{2.2}$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N$ adds self-connections, $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$, $\mathbf{W}^{(l)}$ is a trainable feature map, and σ is a nonlinearity (typically ReLU). The normalized operator $\hat{A} = \tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$ aggregates neighborhood information while controlling scale; a two-layer GCN can be written as

$$f(\mathbf{A}, \mathbf{X}) = \mathbf{H}^{(2)} = \sigma\left(\hat{A}\mathbf{H}^{(1)}\mathbf{W}^{(1)}\right). \tag{2.3}$$

In traffic applications, this aggregation encodes spatial dependence over the graph defined by $\mathbf{A}$; temporal dependence is added by the recurrent component described next.

2.3 Temporal Graph Convolutional Network

T-GCN [6] combines GCN with a GRU for traffic forecasting. At each time t, spatial features $\mathbf{Z}_t = f(\mathbf{A}, \mathbf{X}_t)$ are obtained from (2.3); the GRU then updates a hidden state

$\mathbf{S}_t$ using update and reset gates ($\mathbf{u}_t$, $\mathbf{r}_t$) and trainable matrices $\mathbf{W}_u, \mathbf{W}_r, \mathbf{W}_c$. Training minimizes the discrepancy between predicted speeds $\widehat{Y}_t$ and observed speeds Y_t, for example

$$loss = \|Y_t - \widehat{Y}_t\| + \lambda L_{reg}, \tag{2.4}$$

with L_{reg} an ℓ_2 penalty. T-GCN therefore models spatial dependence through graph convolution and temporal dependence through gating, and is the principal spatiotemporal backbone in our experiments. The non-recurrent GCN backbone applies graph convolution over the input window and is used as a spatial counterpart.

2.4 Graph Structure Learning

Graph structure learning estimates $\mathbf{A}$ (or a weighted coefficient matrix W) from observed data rather than from a predefined map [36]. In traffic, the motivation is that proximity is an imperfect proxy for functional co-movement, and that sparse data-driven graphs may avoid the oversmoothing induced by dense physical adjacencies.

Score-based continuous DAG learners replace combinatorial search over graphs with smooth optimization on real matrices. NOTEARS [10] introduces a differentiable acyclicity constraint; DAGMA [11] strengthens the approach with a log-determinant characterization suitable for efficient optimization. We use DAGMA with a linear structural-equation score and an ℓ_1 sparsity penalty. After fitting, a binary adjacency is obtained by thresholding absolute coefficient magnitudes. Acyclicity here is an optimization device; we do not interpret the learned graph as a causal traffic model. The precise contemporaneous and multi-lag constructions, including thresholds and edge budgets, are specified in Section 3.

Compared with attention mechanisms that reweight neighbors on a fixed topology, GSL changes the edge set itself. Compared with purely temporal models, it makes the spatial operator an object of estimation. In this paper we study that estimated operator both as a single contemporaneous graph and as lag-specific blocks, and we evaluate it against physical and graph-free references under the same forecasting protocol.

2.5 Position of this Work

This work remains a study of GSL for traffic prediction. Relative to the standard GCN/T-GCN pipeline with a physical adjacency, we ask whether a learned adjacency is preferable, and under what conditions. Three extensions relative to a minimal GSL baseline are essential for answering that question honestly: (i) a graph-free control, without which learned graphs cannot be separated from the effect of removing a harmful default; (ii) an explicit multi-lag construction, so that temporal organization of the learned structure is defined by the estimator's input rather than asserted post hoc; and (iii) an analysis of how multi-lag graphs are consumed by recurrent versus non-recurrent backbones. These elements do not change the paper's identity as a graph-structure-learning study; they specify the conditions under which learned structure helps.

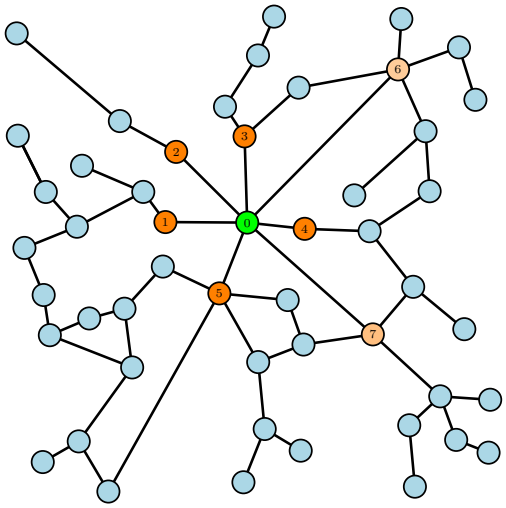


Fig. 2: Conceptual two-dimensional traffic network. Association between a target node (green) and other nodes depends not only on spatial proximity (lighter color indicates larger distance) but also on topology and travel time. The learned adjacency aims to reflect such data-driven associations rather than geographic distance alone.

3 The Proposed Method: Estimating the Adjacency Matrix with Graph Structure Learning

In both GCN [22] and T-GCN [6], the adjacency matrix $\mathbf{A}$ that describes spatial relationships among sensors is typically predefined from urban planning or road-network information. Here we estimate $\mathbf{A}$ from traffic data, which is known as *graph structure learning* (GSL) [36]. The goal is to obtain a graph representation that reflects dependency structure present in the observed series rather than structure imposed solely by physical proximity.

3.1 Motivation

In a typical urban traffic network, spatial proximity of roads need not coincide with statistical dependence among their speeds. Distant segments can be strongly correlated through shared demand, while adjacent segments can be weakly related. Propagation delay is also relevant: association between two sensors may appear only at particular temporal lags. Estimating the adjacency directly from the data allows the forecasting model to aggregate from neighbors defined by observed co-movement rather than only by the map.

Figure 2 illustrates a stylized network related to Figure 1. Three points motivate data-driven adjacency: (i) nearby nodes may share more immediate co-movement than distant ones, but this need not hold for every pair; (ii) associations between distant nodes may occur at longer lags; (iii) some physically adjacent links may carry little predictive information for the target sensor. By learning $\mathbf{A}$ from traffic observations,

the model can place edges according to estimated dependency rather than geographic distance alone.

Throughout this section, $\mathbf{A} \in \{0,1\}^{N\times N}$ denotes the binary adjacency supplied to the forecasting backbone (Eqs. (2.2)–(2.3)), and $W \in \mathbb{R}^{d\times d}$ denotes the continuous coefficient matrix estimated by the graph-learning procedure. The two symbols are not interchangeable: W is the estimator's internal representation; $\mathbf{A}$ is the thresholded graph used by GCN or T-GCN.

3.2 Integration of Learned Structure with Temporal Modeling

The overall pipeline separates three stages:

- **Graph estimation (GSL).** A static adjacency is estimated from training traffic data (contemporaneous or multi-lag constructions below).
- **Normalization.** Every adjacency used in this paper — physical, identity (graph-free control), and learned — is passed through the same GCN-style operator

$$\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2}\,\tilde{\mathbf{A}}^{\top}\,\tilde{\mathbf{D}}^{-1/2}, \qquad \tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}, \quad \tilde{D}_{ii} = \sum_{j} \tilde{A}_{ij}. \tag{3.1}$$

  For the identity adjacency this reduces to $\hat{\mathbf{A}} = \mathbf{I}$, so the graph-free control differs only in the absence of off-diagonal structure.
- **Forecasting backbone.** GCN or T-GCN consumes the normalized operator. The learned graph is *static* after estimation: it is fitted once on the training split. Temporal variation in the forecast is modeled by the recurrent backbone (and, in one variant, by a gate that selects among fixed lag-specific graphs); the edge set itself does not change over time.

Two reference adjacencies accompany the learned graphs in all experiments: the *physical* road-network graph (standard T-GCN/GCN baseline) and a *graph-free* identity control (T-GCN-NoSpatial / GCN-NoSpatial). The latter is required to measure whether learned structure adds value beyond removing a possibly harmful dense proximity graph.

3.3 Continuous Optimization for Graph Structure Learning

Score-based GSL seeks a weighted adjacency that fits the data under a structural equation model (SEM). Following the continuous formulation of Zheng et al. [10], let $\mathbf{X} \in \mathbb{R}^{n\times d}$ be a data matrix with n observations of d variables. In traffic, rows of $\mathbf{X}$ are time indices and columns are sensors. Instead of searching the discrete DAG space, one optimizes over $W \in \mathbb{R}^{d\times d}$ with a smooth acyclicity constraint [10]:

$$\begin{aligned} \min_{W} \quad & F(W) \\ \text{subject to} \quad & h(W) = 0, \end{aligned} \tag{3.2}$$

where F measures data fit and h vanishes exactly on acyclic graphs. NOTEARS uses

$$h(W) = \operatorname{tr}\big(e^{W \circ W}\big) - d, \tag{3.3}$$

with gradient $\nabla h(W) = (e^{W \circ W})^\top \circ 2W$, and solves the program with an augmented Lagrangian

$$L^\rho(W, \alpha) = F(W) + \tfrac{\rho}{2}\,|h(W)|^2 + \alpha\, h(W). \tag{3.4}$$

After optimization, a binary support is obtained by absolute-magnitude thresholding, $\widehat{W} = W \circ \mathbf{1}(|W| > \omega)$. Algorithmic details appear in [10].

Estimator used in this paper (DAGMA).

In the experiments we use **DAGMA** [11], which replaces the NOTEARS trace-exponential constraint with a log-determinant acyclicity characterization suitable for efficient constrained optimization, together with a linear SEM score (ℓ_2 reconstruction loss) and an ℓ_1 sparsity penalty $\lambda_1\|W\|_1$. Acyclicity is an *optimization regularizer* of the estimator. We interpret the learned graph as a statistical dependency structure under the linear SEM assumptions, not as a causal traffic model. DAGMA is applied in two related ways: a *contemporaneous* construction (single graph among sensors; Section 3.4) and a *multi-lag* construction (lag-specific blocks; Section 3.5).

3.4 Contemporaneous Graph Learning (GSL and cGSL)

The contemporaneous construction is a single learned graph among the N sensors from simultaneous observations — the direct analogue of the original GSL-for-traffic idea.

Input and fit.

DAGMA is fitted to the training-split snapshot matrix

$$\mathbf{X} = \text{train\_norm}[\,0 :: \text{PH}\,],$$

i.e. every PH-th training row (one simultaneous vector of N sensor speeds per row; no lag blocks). For Los-loop the row counts are 1612, 806, 538, and 403 for $\text{PH} = 1, \ldots, 4$; for SZ-Taxi, 2380, 1190, 794, and 595. The sparsity coefficient is $\lambda_1 = 0.02$ (Los-loop) and $\lambda_1 = 0.01$ (SZ-Taxi). DAGMA's internal magnitude threshold is 0.3. The binary adjacency is

$$\mathbf{A}_{\text{GSL}} = \mathbf{1}\big(|W| > 0\big)$$

after that internal threshold (absolute-magnitude rule; diagonal removed), using training data only. Edge counts are 28 (Los-loop) and 8 (SZ-Taxi) at every horizon.

cGSL (symmetrized GSL).

The cyclic variant cGSL is not a second fit. It is the binary symmetrization of the same stored GSL artifact:

$$\mathbf{A}_{\text{cGSL}} = \mathbf{1}\big(\mathbf{A}_{\text{GSL}} + \mathbf{A}_{\text{GSL}}^\top > 0\big), \tag{3.5}$$

with the diagonal removed (56 directed entries on Los-loop, 16 on SZ-Taxi). The same artifacts serve both GCN and T-GCN counterparts, so any difference between backbones is not due to a different learned edge set.

Interpretation.

The contemporaneous graph is estimated from simultaneous snapshots. An edge is a statistical association among present observations; it does not encode "sensor j at time t predicts sensor i at time $t+1$." Temporal organization of learned structure is introduced explicitly in the multi-lag construction next, rather than inferred from this DAG.

3.5 Multi-Lag Graph Learning

A single contemporaneous graph cannot distinguish associations at different temporal scales. We therefore estimate lag-specific structure from an explicitly lag-stacked input, still within the same DAGMA/GSL framework.

With lag depth $L = 3$,

$$Z = \big[\, x(t-3),\, x(t-2),\, x(t-1),\, x(t) \,\big],$$

stacked over training windows (1609 windows on Los-loop; 2377 on SZ-Taxi). The variable dimension is $(L+1)N$: 828 (Los-loop) or 624 (SZ-Taxi). DAGMA is fitted *once* on the training split; the artifact is shared across PH $= 1, \ldots, 4$ (unlike the contemporaneous input, which is PH-subsampled).

Here $\lambda_1 = 0.01$ on both datasets, the internal threshold is 0.0 (raw weights retained), and the forecasting consumer thresholds at $|W| > 0.1$ (absolute magnitude; diagonal removed). In the DAGMA convention, W_{ij} quantifies estimated dependence of variable j on variable i; the block mapping lag-k variables to the current-time block yields lag adjacencies $\mathbf{A}_1, \mathbf{A}_2, \mathbf{A}_3$. The contemporaneous block inside W is not consumed by the forecasting methods.

After consumer thresholding, per-lag off-diagonal counts are 12/3/15 (sum 30 slots; union 28 distinct edges) on Los-loop and 0/0/2 (union 2) on SZ-Taxi. The lag-structured reading is consistent with the construction; we do not independently validate it as a physical propagation model, and acyclicity remains a fitting constraint only. The graphs are static after fitting.

3.6 Using Learned Graphs in GCN and T-GCN

Single-graph use (GSL / cGSL / physical / identity).

One normalized adjacency is applied for the whole input window in GCN, and at every recurrent step in T-GCN. This is the standard integration of a learned static structure with temporal modeling: GSL provides persistent spatial dependency estimates; the backbone models temporal evolution on that fixed operator.

Multi-lag use in T-GCN.

When three lag graphs are available, T-GCN can apply them in different ways. We evaluate:

- **Fixed lag assignment (T-GCN-MultiGSL).** At input index t ($t = 0$ most recent), the graph index is

$$\text{graph\_idx} = (T - 1 - t) \bmod 3, \tag{3.6}$$

  so recent steps use lag-1 and older steps cycle through lag-2 and lag-3. No extra parameters.
- **Global mixture (T-GCN-MultiGSL-Weighted, supplementary).** Three learned scalars (softmax) mix the normalized lag operators for every step (+3 parameters).
- **Per-node gate (T-GCN-MultiGSL-Mix).** At each step t and node, a small MLP on $[x_t; h_{t-1}]$ produces a softmax over the $K = 3$ graphs; the mixed operator is applied before the GRU update (+4,419 parameters). Mix changes how the *static* lag graphs are used per node and timestep; it does not relearn the edge set.

Multi-lag use in GCN (GCN-MultiGSL).

The non-recurrent GCN applies a single operator to the window, so lag graphs are merged into a static union

$$\mathbf{A}_{\cup} = \bigvee_{k=1}^{3} \mathbf{A}_k$$

(element-wise OR), then normalized as in (3.1). Comparing GCN-MultiGSL with T-GCN-MultiGSL asks whether the same multi-lag artifacts are useful when consumed as one static graph versus per timestep. Because the backbones also differ, this comparison is evidence about consumption under different architectures, not a fully isolated causal test of consumption alone.

Capacity.

Within a backbone family, graph sources add no trainable parameters. Verified counts (excluding the shared output layer) are 768 for GCN and 12,672 for single-graph T-GCN; Mix adds 4,419 relative to T-GCN-MultiGSL. Capacity-matched checks are reported with the experiments (Section 4).

3.7 Summary of Graph Sources

Table 1 summarizes the adjacencies compared in this paper. The experimental study asks whether replacing the physical graph with a learned one improves forecasting, whether a graph-free control changes that assessment, and — for multi-lag structure — how the learned graphs should be used inside the backbone.

Table 1: Graph sources used in the experiments. All are normalized by Eq. (3.1). "Static" means the adjacency is fixed after estimation; temporal modeling is in the backbone (and in Mix, in the *use* of fixed lag graphs).

Name	Origin	Construction	Use
Physical	road network	dataset adjacency	static
NoSpatial	none	identity $\mathbf{I}$	static
GSL	DAGMA	contemporaneous	static
cGSL	DAGMA	(3.5) of GSL	static
MultiGSL family	DAGMA	lag blocks $\mathbf{A}_{1:3}$	per-step / gate / union

4 Experimental Setup

This section describes the datasets, protocol, model configurations, and evaluation used in our experiments. The implementation in PyTorch, together with data and configuration files, is available at https://github.com/mamintoosi/TGCN-GSL-PyTorch. A preliminary version of the GSL-for-traffic idea was presented in [37].

4.1 Datasets

We use two widely used traffic benchmarks.

SZ-Taxi comprises taxi trajectory speeds in the Luohu District of Shenzhen, China (January 2015), from 156 road sensors aggregated at 15-minute intervals (2976 timesteps).

Los-loop comprises loop-detector speeds on Los Angeles County highways (March 1–7, 2012), from 207 sensors aggregated every 5 minutes (2016 timesteps, seven days).

Each dataset provides a feature matrix of speeds over time and a road-network adjacency used as the physical baseline (Section 3). The two benchmarks contrast urban taxi traffic with highway dynamics.

4.2 Data Split and Normalization

Each series is split chronologically into 80% training and 20% test segments, preserving temporal order: 1612/404 timesteps for Los-loop and 2380/596 for SZ-Taxi. Input windows and targets are generated separately within each segment, so no window crosses the train/test boundary.

Features are normalized by dividing by the *training-split* maximum only (70.0 on Los-loop; 86.4292 on SZ-Taxi). The test split is not used for normalization or for graph learning.

4.3 Forecasting Protocol

The historical window is $T = 12$ steps. Prediction horizons are PH $\in \{1, 2, 3, 4\}$ steps of the native sampling interval: 5–20 minutes on Los-loop and 15–60 minutes on SZ-Taxi. Targets are the multi-step speed vectors produced by a shared linear output

layer (Section 3). Horizon indices are comparable within, not across, datasets because the sampling intervals differ.

Following [6], we focus the comparison on GCN and T-GCN as foundational backbones rather than an exhaustive set of recent architectures: the goal is to assess the effect of graph structure learning inside established frameworks that many later models build upon [5, 38].

4.4 Model Configurations

Within each backbone family we compare physical, graph-free, and learned adjacencies, and (for multi-lag graphs) different ways of using the learned structure (Section 3).

T-GCN family.

T-GCN (physical graph); T-GCN-NoSpatial (identity, graph-free control); T-GCN-GSL and T-GCN-cGSL (contemporaneous learned graphs); T-GCN-MultiGSL, T-GCN-MultiGSL-Weighted, and T-GCN-MultiGSL-Mix (multi-lag graphs under fixed, global-mixture, and per-node gating).

GCN family.

GCN (physical); GCN-NoSpatial; GCN-GSL; GCN-cGSL; and GCN-MultiGSL (static union of the multi-lag graphs).

Architectures within a family are identical apart from the adjacency and, for the multi-lag T-GCN variants, the lag-graph use (Section 3.6). Contemporaneous and multi-lag DAGMA hyperparameters and thresholds are as specified in Sections 3.4 and 3.5. For the contemporaneous construction, the sparsity coefficient is $\lambda_1 = 0.01$ on SZ-Taxi and $\lambda_1 = 0.02$ on Los-loop, consistent with the original GSL protocol.

4.5 Training Protocol

All forecasting models are trained with Adam (learning rate 10^{-3}, weight decay 10^{-4}), batch size 128, for 50 epochs, hidden dimension 64, and a shared linear output map from the hidden state to the PH-dimensional target.

The T-GCN family uses an ℓ_2 loss with an ℓ_2 penalty on weights ($\lambda_{\text{reg}} = 1.5\times10^{-3}$), matching the reference implementation; the GCN family uses mean squared error.

Each configuration is trained under five seeds $\{42, 43, 44, 45, 46\}$. Learned graphs are fitted once and are deterministic (DAGMA-linear, seed 42); the same graph artifacts are shared across forecasting seeds, so seed variability reflects model training only. Experiments were run in PyTorch on a workstation with an NVIDIA RTX 3090 GPU.

4.6 Evaluation Metrics

We report root mean squared error (RMSE, primary) and mean absolute error (MAE), computed on the de-normalized speed scale over the full test set:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(Y_i - \widehat{Y}_i\right)^2}, \tag{4.1}$$

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|Y_i - \widehat{Y}_i\right|. \tag{4.2}$$

Additional diagnostics used in the code base (relative accuracy and R^2) are not required for the comparisons below.

4.7 Statistical Reporting

Main-text numbers are five-seed means with sample standard deviations (ddof = 1). We also report paired per-seed win counts and paired t-tests for reference. With $n = 5$, the exact Wilcoxon signed-rank test cannot fall below $p = 0.0625$; we therefore avoid definitive significance claims.

4.8 Matched-Sparsity and Capacity Controls

To address the concern that any gain from a learned graph might be due only to sparsity (or to extra parameters of a gated variant), we include two scope-labeled controls on Los-loop at PH = 1.

Matched sparsity. At a 30-edge budget we compare RandTop30 (random directed edges, redrawn per seed), CorrTop30 (top-30 training Pearson correlations), and the DAGMA multi-lag graphs under fixed and gated use, with five seeds under the same training protocol. No sparsified physical graph and no full λ/threshold sweep were run.

Capacity. A T-GCN-NoSpatial model with hidden dimension 74 (16,872 parameters) more closely matches the gated multi-lag model (17,091) than standard T-GCN-NoSpatial at hidden 64 (12,672). This check is single-seed and is labeled as such.

4.9 Ablation Studies

In addition to the main comparison of graph sources, we examine:

- **Physical vs. learned vs. graph-free.** The physical adjacency is compared with contemporaneous GSL/cGSL and with the identity control, so that learned structure is not credited solely for removing a dense proximity graph.
- **GSL vs. cGSL.** Directed versus symmetrized use of the *same* contemporaneous artifact on each backbone.
- **Multi-lag structure and its use.** Fixed lag assignment, global weights, per-node gating, and static union consumption of the same multi-lag artifacts (Section 3.6).

- **Temporal resolution.** A 15-minute resampling of Los-loop (Section 4.10).

4.10 Temporal-Resolution Variant

Los-loop is additionally averaged over consecutive triplets of 5-minute observations, yielding a 15-minute series of length $T = 672$ with an 80/20 split. Horizons PH $= 1, \ldots, 4$ then correspond to 15–60 minutes ahead. This is a resolution variant of Los-loop, not a third dataset, and its PH indices are not equivalent to those of the 5-minute series. A separate multi-lag DAGMA fit is used ($\lambda_1 = 0.01$; consumer threshold $|W| > 0.1$); T-GCN-NoSpatial, T-GCN-MultiGSL, and T-GCN-MultiGSL-Mix are trained with five seeds under the standard protocol.

5 Results

We evaluate whether learning the adjacency from traffic data improves forecasting relative to a physical road-network graph, and under what conditions. Unless noted otherwise, each value is the mean test RMSE over five training seeds with the sample standard deviation in parentheses; learned graphs are fitted once and shared across seeds. Improvements are relative RMSE reductions with respect to the named reference. Statistical qualifications follow Section 4.7.

5.1 Baselines: Physical Graph versus Graph-Free Control

Table 2 reports the T-GCN family on both benchmarks. The physical road-network adjacency is the weakest configuration in its family in every dataset–horizon cell. On Los-loop at PH1, T-GCN obtains 7.88 ± 0.28 RMSE, while the graph-free control T-GCN-NoSpatial obtains 5.25 ± 0.19 — a 33.3% relative reduction when the physical graph is removed. The same direction holds at PH2–PH4 on Los-loop (29.2%, 27.0%, 24.1%) and at every horizon on SZ-Taxi (24.4%–25.2%). NoSpatial beats Physical in 5/5 seeds in all eight cells.

The GCN family (Table 3) shows the same pattern (Los-loop PH1: GCN 8.14 ± 0.11 vs. GCN-NoSpatial 4.88 ± 0.31).

These baselines matter for evaluating GSL: an improvement over the physical graph alone does not establish that learned structure is useful, because removing the graph already reduces error substantially. Figure 5 contrasts the dense physical adjacency with the much sparser multi-lag learned union on Los-loop.

5.2 Contemporaneous Learned Graphs (GSL and cGSL)

Replacing the physical adjacency with a single contemporaneous DAGMA graph (Section 3.4) improves on Physical in most T-GCN cells but does not surpass the graph-free control.

On Los-loop PH1, T-GCN-GSL and T-GCN-cGSL obtain 5.86 ± 0.21 and 5.82 ± 0.23, about 26% better than Physical yet about 11% worse than NoSpatial. The ordering Physical $>$ single learned graph $>$ NoSpatial (in error) holds at all Los-loop horizons and on SZ-Taxi: neither contemporaneous learned graph beats the graph-free control in any dataset$\times$horizon cell.

For GCN, cGSL is clearly better than the directed GSL on Los-loop (5.76 ± 0.20 vs. 7.83 ± 0.11 at PH1) but still worse than GCN-NoSpatial. For T-GCN, GSL and cGSL are nearly indistinguishable. Symmetrization therefore matters mainly for the non-recurrent backbone that applies a single symmetric-style aggregation; this is an empirical observation, not a general rule.

In short, a single learned contemporaneous graph recovers only part of the error reduction obtained by discarding the physical adjacency.

5.3 Is the Gain Only Sparsity?

Table 4 reports the matched-sparsity control on Los-loop PH1. At a common budget of 30 directed edges, a random graph (RandTop30) obtains 6.10 ± 0.12 and a top-correlation graph (CorrTop30) obtains 5.39 ± 0.10 — both worse than NoSpatial (5.25 ± 0.19). The DAGMA multi-lag graphs at the same budget obtain 4.84 ± 0.11 under fixed per-step use and 4.49 ± 0.14 under gated use (Section 5.4). Thus, on this cell, sparsity alone does not explain the multi-lag gain; where the edges are placed matters. The conclusion is limited to Los-loop PH1.

A capacity-matched graph-free model (hidden 74; single seed; lower block of Table 4) stays at about 5.14 RMSE, so the extra parameters of the gated variant do not by themselves account for its error level.

5.4 Multi-Lag Learned Graphs

The multi-lag construction estimates lag-specific graphs (Section 3.5). On Los-loop these graphs are informative (12/3/15 edges by lag; union 28). Consuming them with a fixed lag-aligned assignment in T-GCN (T-GCN-MultiGSL) yields 4.84 ± 0.11 at PH1: better than NoSpatial by 7.8% and better than Physical by 38.5%. Per-node gating (T-GCN-MultiGSL-Mix) improves further to 4.49 ± 0.14.

Relative to the graph-free control, Mix reduces RMSE by 14.5%, 11.9%, 9.2%, and 10.9% at PH1–PH4 on Los-loop (Table 2; 5/5 paired seeds at every horizon). Relative to the physical T-GCN baseline, the reductions are 43.0%, 37.6%, 33.7%, and 32.3%. Figure 4 shows the per-seed distributions: Mix is the strongest Los-loop configuration at every horizon. The global-weight variant (Weighted) is essentially tied with the fixed assignment (differences ≤ 0.01 RMSE).

When the *same* multi-lag artifacts are merged into one static union and consumed by non-recurrent GCN (GCN-MultiGSL), error is 9.78 ± 0.14 on Los-loop PH1 — worse than the physical GCN and more than twice the T-GCN-MultiGSL error on the same graphs (Figure 3). The union retains 28 of the 30 lagged edge slots. Hence the edge set alone does not determine usefulness: how the learned multi-lag structure is used inside the backbone is a major factor. Because GCN and T-GCN also differ architecturally, this is not a fully isolated test of consumption, but it shows that learned multi-lag graphs are not automatically beneficial once estimated.

5.5 Dataset Dependence and Temporal Resolution

SZ-Taxi.

On SZ-Taxi the multi-lag fit retains only two lagged edges. All learned variants remain close to the graph-free control: Mix vs. NoSpatial differs by at most 0.015 RMSE (relative reductions $\leq 0.34\%$), with Mix winning 4/5, 4/5, 5/5, and 4/5 paired seeds at PH1–PH4. The strong Los-loop multi-lag result therefore does not transfer to SZ-Taxi under the present construction.

Los-loop at 15-minute sampling (resolution control).

SZ-Taxi is sampled every 15 minutes, whereas Los-loop is sampled every 5 minutes. To test whether the weak SZ-Taxi separation could be an artifact of coarser temporal resolution rather than of the dataset itself, we resampled Los-loop to 15-minute steps by averaging consecutive triplets (Section 4.10) and reran the multi-lag comparison on that series.

At 15 minutes ahead, NoSpatial obtains 8.600 ± 0.249, MultiGSL 7.246 ± 0.298, and Mix 6.240 ± 0.187 — a 27.4% relative reduction vs. NoSpatial with 5/5 seed wins, *larger* than the 14.5% obtained at 5-minute sampling. Relative reductions remain 21.4%, 19.8%, and 16.1% at 30, 45, and 60 minutes (Table 5). Absolute RMSE values are not comparable across sampling intervals, and PH indices of the variant are not those of the 5-minute series.

Because Los-loop at 15-minute sampling improves *more* than at 5-minute sampling, while SZ-Taxi at the same 15-minute interval stays near the graph-free baseline, temporal resolution alone does not explain the SZ-Taxi result. The contrast instead points to dataset-specific factors, including how much lag structure the estimator recovers (30 lagged slots on 5-minute Los-loop, 32 on 15-minute Los-loop, versus only 2 on SZ-Taxi).

5.6 Summary of Findings

1. Learning $\mathbf{A}$ from data (GSL) improves on the physical graph in many settings, but a graph-free control is necessary to judge the true contribution of spatial structure.
2. A single contemporaneous learned graph does not beat the graph-free control on these benchmarks.
3. Multi-lag learned graphs help on Los-loop when used in a lag-aligned way in T-GCN; gated mixing is the strongest configuration. Sparsity alone does not explain this gain on the matched-budget cell.
4. The benefit is dataset-dependent: SZ-Taxi shows little separation from the graph-free baseline, consistent with a nearly empty multi-lag fit.

Table 2: T-GCN family: test RMSE (mean, sample standard deviation in parentheses) over five seeds. Physical: road-network graph. NoSpatial: identity adjacency. GSL/cGSL: contemporaneous learned graphs. MultiGSL family: multi-lag graphs under fixed, weighted, and gated use. Best (lowest) mean RMSE in each dataset block is in bold.

Method	PH1	PH2	PH3	PH4
Los-loop (5 min)				
T-GCN (Physical)	7.88 (0.28)	8.13 (0.09)	8.37 (0.17)	8.66 (0.16)
T-GCN-NoSpatial	5.25 (0.19)	5.76 (0.08)	6.11 (0.07)	6.58 (0.12)
T-GCN-GSL	5.86 (0.21)	6.30 (0.06)	6.66 (0.12)	7.04 (0.09)
T-GCN-cGSL	5.82 (0.23)	6.35 (0.12)	6.67 (0.11)	7.09 (0.12)
T-GCN-MultiGSL	4.84 (0.11)	5.45 (0.14)	5.94 (0.03)	6.26 (0.03)
T-GCN-MultiGSL-Weighted	4.83 (0.11)	5.44 (0.13)	5.93 (0.03)	6.25 (0.03)
T-GCN-MultiGSL-Mix	**4.49 (0.14)**	**5.08 (0.15)**	**5.55 (0.16)**	**5.86 (0.07)**
SZ-Taxi (15 min)				
T-GCN (Physical)	5.45 (0.11)	5.55 (0.08)	5.60 (0.05)	5.63 (0.06)
T-GCN-NoSpatial	4.12 (0.00)	4.16 (0.01)	4.19 (0.01)	4.23 (0.01)
T-GCN-GSL	4.28 (0.05)	4.31 (0.03)	4.33 (0.01)	4.37 (0.02)
T-GCN-cGSL	4.30 (0.03)	4.34 (0.02)	4.38 (0.03)	4.41 (0.01)
T-GCN-MultiGSL	4.13 (0.02)	4.16 (0.00)	4.20 (0.01)	4.22 (0.00)
T-GCN-MultiGSL-Weighted	4.13 (0.02)	4.16 (0.00)	4.20 (0.01)	4.22 (0.00)
T-GCN-MultiGSL-Mix	**4.12 (0.02)**	**4.15 (0.01)**	**4.18 (0.00)**	**4.22 (0.01)**

6 Discussion

6.1 Why the Physical Graph Can Be Harmful

The physical road-network graph is dense relative to the number of sensors (on the order of 10^3 off-diagonal entries for 207 Los-loop detectors). Under repeated neighborhood aggregation this density can oversmooth node representations, so that a graph-based forecaster is worse than the same backbone with identity adjacency. The consistent gap between Physical and NoSpatial on both benchmarks and both backbones is consistent with that reading. It also changes how GSL should be judged: improving on the physical graph is not sufficient evidence that learned structure is useful, because discarding the graph already helps.

Learned contemporaneous graphs occupy an intermediate position — less harmful than the physical graph, but not better than using no graph. What they appear to provide is a less harmful neighborhood, not enough additional signal to beat identity under these protocols.

6.2 When Learned Structure Helps

The positive Los-loop result is specific: multi-lag DAGMA graphs consumed in a lag-aligned way inside T-GCN. Three observations support a structure-learning

Table 3: GCN family: test RMSE (mean, sample standard deviation in parentheses) over five seeds. GCN-MultiGSL consumes the union of the three multi-lag graphs as one static adjacency. Best (lowest) mean RMSE in each dataset block is in bold.

Method	PH1	PH2	PH3	PH4
Los-loop (5 min)				
GCN (Physical)	8.14 (0.11)	8.54 (0.28)	8.76 (0.26)	8.76 (0.29)
GCN-NoSpatial	**4.88 (0.31)**	**5.60 (0.39)**	**6.02 (0.28)**	**6.26 (0.26)**
GCN-GSL	7.83 (0.11)	8.48 (0.21)	8.45 (0.19)	8.93 (0.12)
GCN-cGSL	5.76 (0.20)	6.33 (0.32)	6.68 (0.25)	6.88 (0.21)
GCN-MultiGSL	9.78 (0.14)	10.05 (0.16)	10.24 (0.13)	10.27 (0.11)
SZ-Taxi (15 min)				
GCN (Physical)	5.96 (0.01)	5.98 (0.00)	5.99 (0.00)	6.00 (0.00)
GCN-NoSpatial	**4.11 (0.00)**	**4.15 (0.00)**	**4.19 (0.00)**	**4.22 (0.00)**
GCN-GSL	4.88 (0.01)	4.91 (0.00)	4.93 (0.01)	4.96 (0.00)
GCN-cGSL	4.64 (0.00)	4.67 (0.00)	4.70 (0.00)	4.73 (0.00)
GCN-MultiGSL	4.82 (0.01)	4.85 (0.00)	4.88 (0.01)	4.90 (0.00)

Table 4: Matched-sparsity and capacity controls on Los-loop at PH1 only. Five seeds for graph rows; capacity rows are single-seed (seed 42).

Configuration	RMSE	Budget / params	vs NoSpatial
RandTop30	6.10 (0.12)	30 random edges	+16.1%
CorrTop30	5.39 (0.10)	30 top-|Pearson|	+2.6%
T-GCN-NoSpatial	5.25 (0.19)	identity	—
T-GCN-MultiGSL	4.84 (0.11)	30 DAGMA lag slots	−7.8%
T-GCN-MultiGSL-Mix	4.49 (0.14)	same graphs, gated	−14.5%
NoSpatial h=64 (seed 42)	5.14	12,672 params	—
NoSpatial h=74 (seed 42)	5.14	16,872 params	$\approx 0\%$
Mix (seed 42)	4.46	17,091 params	−13.3%

interpretation rather than a pure sparsity interpretation. First, at a matched 30-edge budget, random and correlation-placed graphs are worse than no graph, while DAGMA-placed lag graphs are better (Los-loop PH1 only). Second, the multi-lag construction recovers substantially more lagged structure on Los-loop than on SZ-Taxi (30 vs. 2 slots). Third, the same multi-lag artifacts fail when merged into a static union for GCN, so estimation alone is not enough; the backbone must use the structure in a way that matches how it was organized (lag blocks).

The GCN/T-GCN contrast for contemporaneous graphs (cGSL helps GCN more than T-GCN) is an aggregation-compatibility observation of this study. We do not interpret the contemporaneous DAG as a temporal graph (Section 3.4); temporal organization of learned structure is carried by the multi-lag construction only.

Table 5: Los-loop temporal-resolution variant at 15-minute sampling. PH k denotes $15k$ minutes ahead and is not comparable to PH k of the 5-minute series. Five seeds.

Method	PH1 (15 min)	PH2 (30 min)	PH3 (45 min)	PH4 (60 min)
T-GCN-NoSpatial	8.600 (0.249)	9.311 (0.197)	10.048 (0.074)	10.460 (0.196)
T-GCN-MultiGSL	7.246 (0.298)	8.571 (0.327)	9.112 (0.077)	9.756 (0.210)
T-GCN-MultiGSL-Mix	**6.240 (0.187)**	**7.322 (0.338)**	**8.063 (0.203)**	**8.777 (0.326)**
Mix vs NoSpatial	−27.4%	−21.4%	−19.8%	−16.1%
Mix wins (of 5)	5	5	5	5

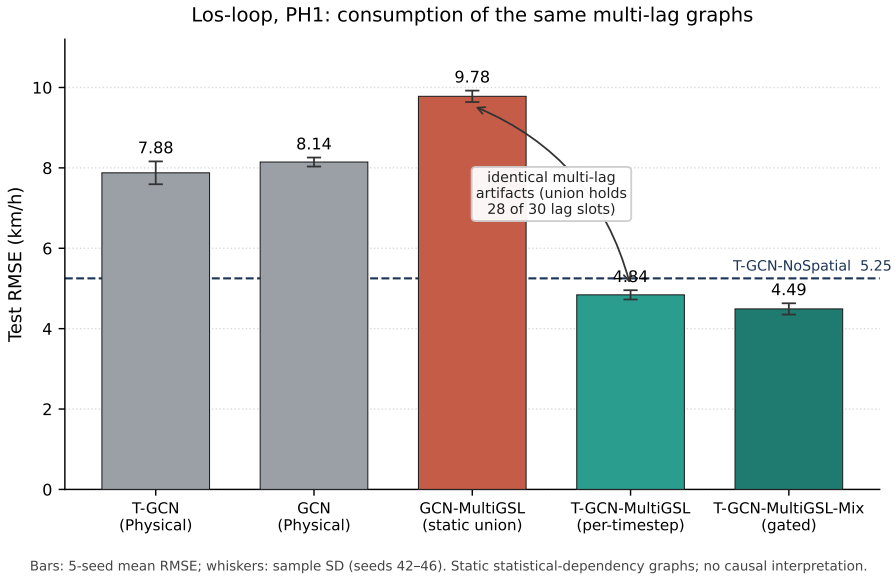


Fig. 3: Los-loop PH1: the same multi-lag DAGMA artifacts under static-union consumption (GCN-MultiGSL) versus per-timestep use (T-GCN-MultiGSL), with physical and gated references. Bars are five-seed mean RMSE; whiskers are sample standard deviations. The union retains 28 of 30 lagged edge slots. Graphs are statistical dependency estimates, not causal maps.

6.3 What the Learned Graph Represents

We treat the learned adjacencies as *statistical dependency* estimates under a linear DAGMA fit. Acyclicity is an optimization constraint, not evidence of causal traffic mechanisms. The lag-structured reading of the multi-lag blocks is consistent with the input construction $Z = [x(t-3), \ldots, x(t)]$ and with the consumption experiments, but it is not independently validated against ground-truth propagation. The gated variant

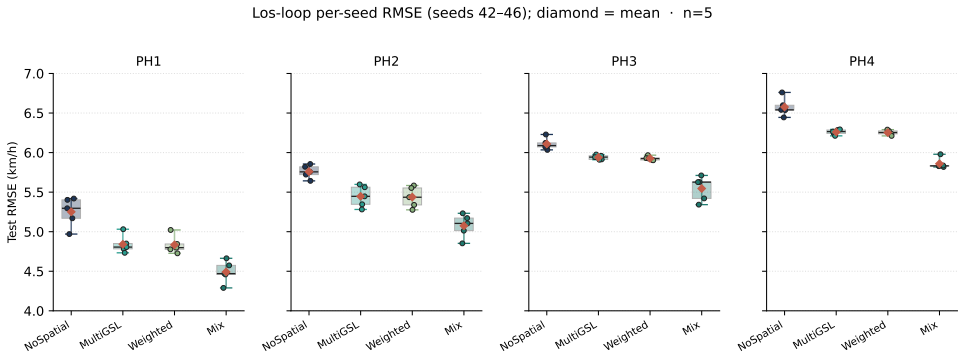


Fig. 4: Los-loop per-seed test RMSE for the multi-lag use ladder (seeds 42–46). Diamonds mark five-seed means matching Table 2.

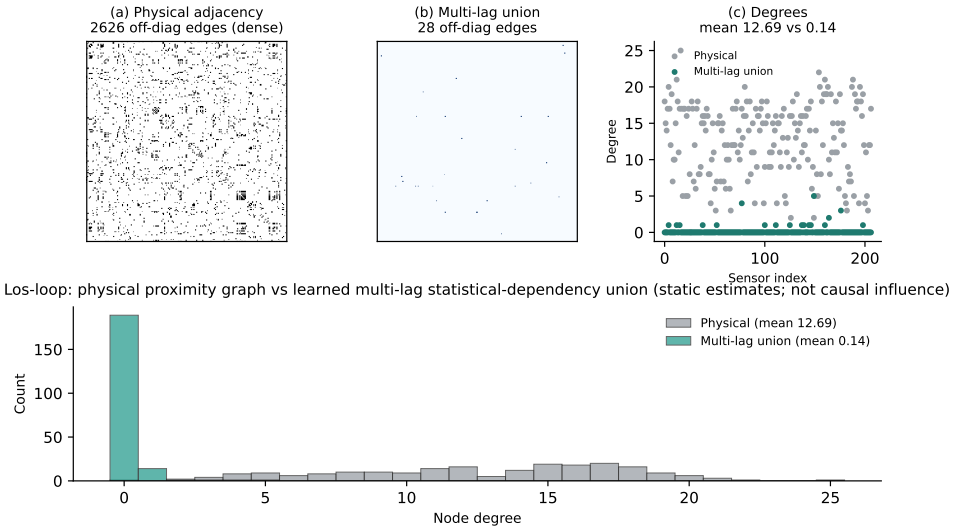


Fig. 5: Los-loop graph structure: (a) physical adjacency (2626 off-diagonal edges; mean degree 12.69); (b) union of multi-lag DAGMA graphs after consumer thresholding (28 edges; mean degree 0.14); (c) node degrees. Learned graphs describe statistical dependency structure.

changes how static lag graphs are used per node and timestep; it does not produce a time-varying edge set.

6.4 Dataset Dependence

SZ-Taxi and Los-loop differ in sampling interval, sensor count, and road environment. The 15-minute Los-loop control (Section 5.5) shows that coarser sampling alone does

not suppress the multi-lag gain. The more concrete difference in these experiments is how much structure the estimator recovers: a nearly empty multi-lag fit on SZ-Taxi is associated with results close to the graph-free baseline. This is a scope condition on GSL for traffic, not a failure of the general idea that adjacency can be estimated from data.

7 Limitations

- **Statistical power.** Main results use five training seeds. Sample standard deviations and paired win counts are reported; with $n = 5$ the exact Wilcoxon test cannot fall below $p = 0.0625$, so we avoid definitive significance claims.
- **Control scope.** Matched-sparsity and capacity controls exist only for Los-loop at PH1. There is no sparsified physical graph, no λ/threshold sweep, and no SZ-Taxi sparsity control.
- **Datasets and horizons.** Two benchmarks; PH ≤ 4 at native sampling. The 15-minute Los-loop study is a resolution control, not evidence for PH5–8 at 5-minute sampling.
- **Estimator assumptions.** Linear DAGMA with ℓ_2 score and acyclicity regularizer; nonlinear dependency and misspecified lag depth $L = 3$ are not examined. Multi-lag fitting is expensive for large N.
- **Static graphs.** Learned adjacencies are fitted once. Mix varies *use* of those graphs, not the edge set; time-varying structure learning remains future work.
- **Backbones.** Evaluation is limited to GCN and T-GCN. Larger attention or transformer traffic models are outside the present study.

8 Conclusion and Future Works

This paper revisits graph structure learning for traffic forecasting under a protocol that includes physical, graph-free, contemporaneous learned, and multi-lag learned adjacencies on two public benchmarks. The main findings are:

1. A dense physical road-network graph can be a poor default for GCN and T-GCN at short horizons; a graph-free identity control is a necessary reference when evaluating learned structure.
2. A single contemporaneous DAGMA graph improves on the physical graph in many settings but does not beat the graph-free control on these datasets.
3. Multi-lag learned graphs help on Los-loop when consumed in a lag-aligned way inside T-GCN; per-node gating (T-GCN-MultiGSL-Mix) is the strongest configuration (14.5% relative RMSE reduction vs. the graph-free baseline at PH1; up to 43.0% vs. the physical baseline across horizons). Matched-budget controls show that sparsity alone does not explain this gain on the studied cell.
4. The benefit is dataset-dependent. On SZ-Taxi, where the multi-lag fit is nearly empty, learned variants remain close to the graph-free baseline. A 15-minute resampling of Los-loop retains a large relative gain, so coarser sampling alone does not explain the SZ-Taxi result.

5. Learned graphs are interpreted as statistical dependency structure, not causal maps of traffic influence.

Future work includes longer horizons at 5-minute resolution, additional networks, sparsified-physical controls and λ/threshold sensitivity, other backbones, hybrid combinations with attention [8], and genuinely time-varying graph estimation (e.g. incremental or sliding-window structure learning) rather than gated use of a fixed graph. Overall, learning the adjacency from traffic data can improve forecasting when the recovered structure is informative and is used in a manner consistent with how it was estimated; it is not a universal replacement for either the physical graph or a graph-free model.

Appendix A Bibliometric Analysis Methodology

To justify the focus on graph-based approaches in traffic forecasting and to trace the evolution of the field, we conducted a systematic bibliometric analysis. This appendix supports the single motivating observation used in the Introduction; it is not part of the forecasting experiments.

A.1 Data Collection and Initial Filtering

A broad search in Scopus with the keyword "traffic forecast*" retrieved approximately 60,000 documents. Filtering for peer-reviewed English articles in Engineering or Computer Science yielded a core collection of 784 articles published between 2000 and 2025.

A.2 Keyword Processing and Semantic Clustering

The raw data contained 6,343 unique keywords. After removing terms with fewer than 10 occurrences, 217 keywords remained. An initial VOSviewer network (Figure A1) showed near-duplicate terms ("graph neural network", "GNN", "graph convolution network") as separate nodes.

A cleaning pipeline in the project repository (preprocessing, expansion of 30 common abbreviations, and Sentence-BERT semantic clustering) condensed these terms into 66 concept groups. Figure A2 shows word clouds for three representative clusters.

A.3 Key Findings

The refined network (Figure A3) shows that graph-based methods are the most recent methodological focus in this literature, while many such methods still rely on predefined graph structures. That observation motivates studying graph structure learning for traffic forecasting.

Appendix B Additional MAE Results

Table B1 reports mean MAE (sample standard deviation in parentheses) over five seeds on the de-normalized scale, parallel to the main RMSE tables.

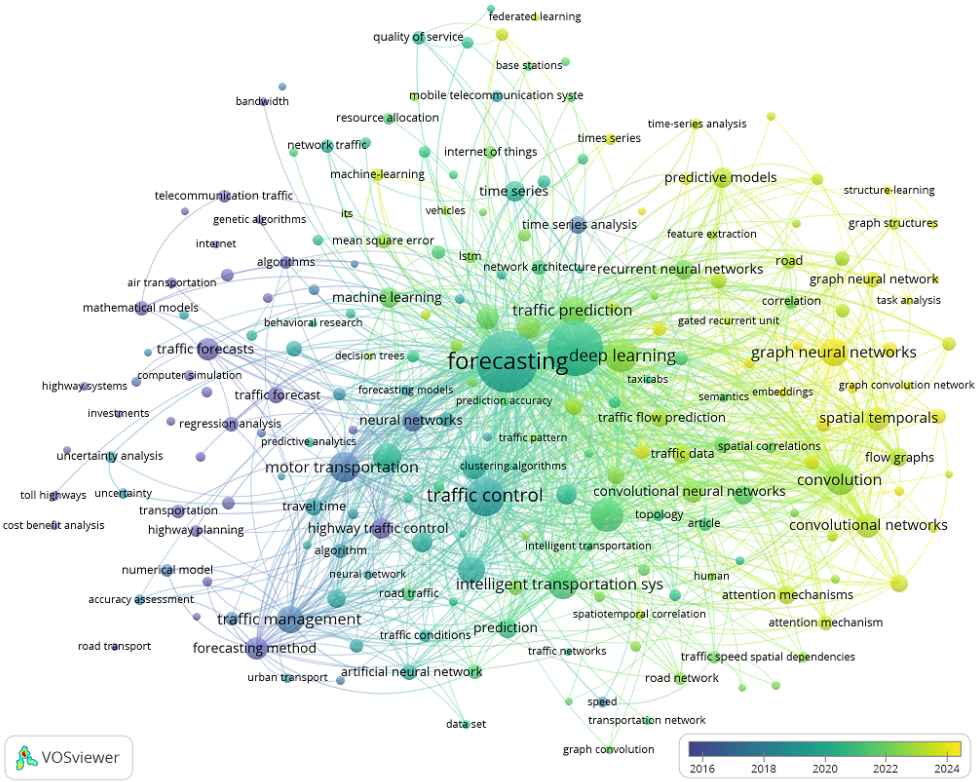
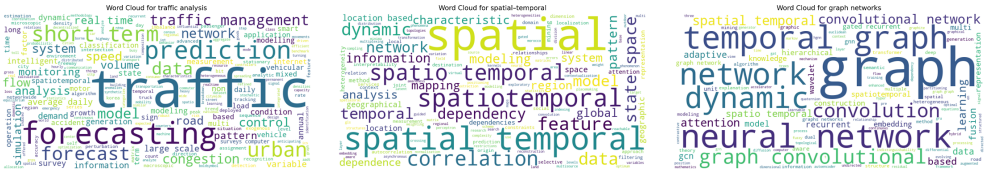


Fig. A1: Initial keyword co-occurrence network (217 keywords) before semantic grouping.



(a) Traffic analysis (b) Spatio-temporal (c) Graph networks

Fig. A2: Word clouds for three bibliometric clusters.

References

[1] Akhtar, M., Moridpour, S.: A review of traffic congestion prediction using artificial intelligence. Journal of Advanced Transportation **2021** (2021)

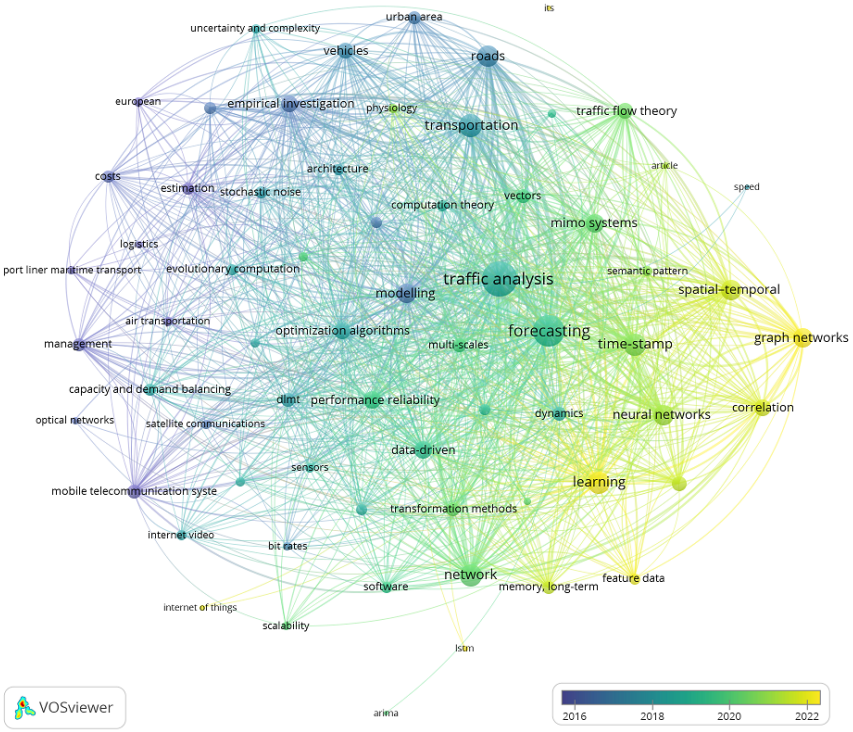


Fig. A3: Clustered keyword co-occurrence network from 784 traffic forecasting articles (2000–2025), colored by average publication year.

[2] Ye, J., Zhao, J., Ye, K., Xu, C.: How to build a graph-based deep learning architecture in traffic domain: A survey. IEEE Transactions on Intelligent Transportation Systems (2020)

[3] Xie, P., Li, T., Liu, J., Du, S., Yang, X., Zhang, J.: Urban flow prediction from spatiotemporal data using machine learning: A survey. Information Fusion **59**, 1–12 (2020)

[4] Chen, F., Zhang, Y., Chen, L., Meng, X., Qi, Y., Wang, J.: Dynamic traveling time forecasting based on spatial-temporal graph convolutional networks. Front. Comput. Sci. **17**(6) (2023) https://doi.org/10.1007/s11704-023-2704-x

[5] Zhou, Z., Basker, R., Yeung, D.-Y.: Graph neural networks for multivariate time-series forecasting via learning hierarchical spatiotemporal dependencies. Engineering Applications of Artificial Intelligence **147**, 110304 (2025) https://doi.org/10.1016/j.engappai.2025.110304

Table B1: Test MAE (mean, sample standard deviation) over five seeds.

Method	PH1	PH2	PH3	PH4
Los-loop				
T-GCN	5.49 (0.21)	5.65 (0.08)	5.75 (0.14)	6.00 (0.16)
T-GCN-NoSpatial	3.06 (0.09)	3.30 (0.06)	3.49 (0.07)	3.72 (0.08)
T-GCN-GSL	3.69 (0.14)	3.91 (0.08)	4.12 (0.03)	4.30 (0.06)
T-GCN-cGSL	3.48 (0.13)	3.76 (0.08)	3.94 (0.09)	4.18 (0.08)
T-GCN-MultiGSL	2.82 (0.06)	3.14 (0.08)	3.35 (0.02)	3.50 (0.05)
T-GCN-MultiGSL-Weighted	2.81 (0.05)	3.13 (0.08)	3.33 (0.03)	3.49 (0.04)
T-GCN-MultiGSL-Mix	2.73 (0.07)	2.99 (0.10)	3.22 (0.09)	3.39 (0.02)
GCN	5.79 (0.03)	6.01 (0.16)	6.12 (0.15)	6.12 (0.15)
GCN-NoSpatial	3.00 (0.21)	3.39 (0.24)	3.55 (0.18)	3.68 (0.22)
GCN-GSL	5.34 (0.08)	5.68 (0.14)	5.65 (0.15)	5.92 (0.07)
GCN-cGSL	3.65 (0.14)	3.97 (0.19)	4.12 (0.16)	4.25 (0.16)
GCN-MultiGSL	6.60 (0.04)	6.76 (0.05)	6.84 (0.08)	6.88 (0.05)
SZ-Taxi				
T-GCN	4.08 (0.05)	4.15 (0.04)	4.20 (0.03)	4.22 (0.02)
T-GCN-NoSpatial	2.76 (0.04)	2.78 (0.03)	2.80 (0.04)	2.83 (0.04)
T-GCN-GSL	2.87 (0.13)	2.89 (0.07)	2.89 (0.05)	2.89 (0.01)
T-GCN-cGSL	2.86 (0.02)	2.92 (0.03)	2.99 (0.08)	2.96 (0.06)
T-GCN-MultiGSL	2.82 (0.08)	2.83 (0.03)	2.85 (0.05)	2.88 (0.03)
T-GCN-MultiGSL-Weighted	2.82 (0.08)	2.83 (0.03)	2.85 (0.05)	2.88 (0.03)
T-GCN-MultiGSL-Mix	2.75 (0.09)	2.77 (0.04)	2.79 (0.03)	2.84 (0.08)
GCN	4.41 (0.00)	4.42 (0.00)	4.43 (0.00)	4.44 (0.00)
GCN-NoSpatial	2.75 (0.03)	2.77 (0.01)	2.79 (0.01)	2.82 (0.01)
GCN-GSL	3.17 (0.01)	3.20 (0.00)	3.23 (0.01)	3.26 (0.01)
GCN-cGSL	3.03 (0.01)	3.06 (0.00)	3.08 (0.00)	3.11 (0.00)
GCN-MultiGSL	3.04 (0.01)	3.08 (0.00)	3.11 (0.00)	3.14 (0.01)

[6] Zhao, L., Song, Y., Zhang, C., Liu, Y., Wang, P., Lin, T., Deng, M., Li, H.: T-GCN: A temporal graph convolutional network for traffic prediction. IEEE Trans. on Intell. Transportation Systems **21**(9), 3848–3858 (2020)

[7] Bai, J., Zhu, J., Song, Y., Zhao, L., Hou, Z., Du, R., Li, H.: A3t-gcn: Attention temporal graph convolutional network for traffic forecasting. ISPRS International Journal of Geo-Information **10**(7) (2021) https://doi.org/10.3390/ijgi10070485

[8] Sun, C., Li, C., Lin, X., Zheng, T., Meng, F., Rui, X., Wang, Z.: Attention-based graph neural networks: a survey. Artificial Intelligence Review **56**(2), 2263–2310 (2023) https://doi.org/10.1007/s10462-023-10577-2

[9] Chen, G., Chen, L., Li, Z., Wei, Y., He, Z., Yang, F.: Ta-stgcn: trend-aware attention spatio-temporal graph convolutional network for traffic flow prediction. International Journal of Data Science and Analytics **22**(1), 56 (2026) https://doi.org/10.1007/s41060-026-01040-w

[10] Zheng, X., Aragam, B., Ravikumar, P., Xing, E.P.: Dags with no tears: Continuous optimization for structure learning. In: NeurIPS (2018)

[11] Bello, K., Aragam, B., Ravikumar, P.: Dagma: learning dags via m-matrices and a log-determinant acyclicity characterization. In: Proceedings of the 36th International Conference on Neural Information Processing Systems. NIPS '22. Curran Associates Inc., Red Hook, NY, USA (2024)

[12] Hamed, M.M., Al-Masaeid, H.R., Said, Z.M.B.: Short-term prediction of traffic volume in urban arterials. Journal of Transportation Engineering **121**(3), 249–254 (1995)

[13] Fabian, X., Ban, G., Boussaïd, R., *et al.*: Modeling and forecasting vehicular traffic flow as a seasonal arima process. Journal of Transportation Engineering **129**(6), 664–672 (2003)

[14] Smola, A.J., Schölkopf, B.: A tutorial on support vector regression. Statistics and Computing **14**(3), 199–222 (2004)

[15] Lippi, M., Bertini, M., Frasconi, P.: Short-term traffic flow forecasting. IEEE Transactions on Intelligent Transportation Systems **14**(2), 871–882 (2013)

[16] Ojeda, L.L., Kibangou, A.Y., Wit, C.C.: Adaptive kalman filtering for multi-step ahead traffic flow prediction. In: American Control Conference (2013)

[17] Park, D., Rilett, L.R.: Forecasting freeway link travel times with a multilayer feedforward neural network. Computer-Aided Civil and Infrastructure Engineering **14**(5), 357–367 (2010)

[18] Zhang, J., Zheng, Y., Qi, D.: Deep spatio-temporal residual networks for citywide crowd flows prediction, 1655–1661 (2017)

[19] Cao, X., Zhong, Y., Zhou, Y., Wang, J., Zhu, C., Zhang, W.: Interactive temporal recurrent convolution network for traffic prediction. IEEE Access **5**, 9893–9901 (2017)

[20] Fu, R., Zhang, Z., Li, L.: Using lstm and gru neural network methods for traffic flow prediction. In: Youth Academic Conference of Chinese Association of Automation, pp. 324–328 (2017)

[21] Khan, H.M., Khan, A., Villar, S.G., Lopez, L.A.D., Almaleh, A., Al-Qahtani, A.M.: A comparative study of optimized-lstm models using tree-structured parzen estimator for traffic flow forecasting in intelligent transportation. Computers, Materials and Continua **83**(2), 3369–3388 (2025) https://doi.org/10.32604/cmc.2025.060474

[22] Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional

networks. In: ICLR (2017)

[23] Zuo, J., Zeitouni, K., Taher, Y., Garcia-Rodriguez, S.: Graph convolutional networks for traffic forecasting with missing values. Data Mining and Knowledge Discovery **37**(2), 913–947 (2023) https://doi.org/10.1007/s10618-022-00903-7

[24] Wang, B., Long, Z., Sheng, J., Zhong, Q.: Spatial–temporal similarity fusion graph adversarial convolutional networks for traffic flow forecasting. Journal of the Franklin Institute **361**(17), 107299 (2024) https://doi.org/10.1016/j.jfranklin.2024.107299

[25] Xia, Z., Zhang, Y., Yang, J., Xie, L.: Dynamic spatial–temporal graph convolutional recurrent networks for traffic flow forecasting. Expert Systems with Applications **240**, 122381 (2024) https://doi.org/10.1016/j.eswa.2023.122381

[26] Remmouche, B., Boukraa, D., Zakharova, A., Bouwmans, T., Taffar, M.: Long-term spatio-temporal graph attention network for traffic forecasting. Expert Systems with Applications **288**, 128244 (2025) https://doi.org/10.1016/j.eswa.2025.128244

[27] Zeb, A., Zheng, J., Ye, Y., Chen, J., Zhang, S., Wei, X., Jianqiao Yu, J.: Traffic forecasting with meta attentive graph convolutional recurrent network. Expert Systems with Applications **287**, 128073 (2025) https://doi.org/10.1016/j.eswa.2025.128073

[28] Jia, C., Jiang, F., Chen, Z., Wang, R., Xiao, Y.: A dynamic traffic flow prediction method based on spatio-temporal graph transformer. Applied Intelligence **55**(16), 1048 (2025) https://doi.org/10.1007/s10489-025-06895-3

[29] Liu, R., Shin, S.-Y.: A review of traffic flow prediction methods in intelligent transportation system construction. Applied Sciences **15**(7) (2025) https://doi.org/10.3390/app15073866

[30] Peng, L., Liao, X., Li, T., Guo, X., Wang, X.: An overview based on the overall architecture of traffic forecasting. Data Science and Engineering **9**(3), 341–359 (2024) https://doi.org/10.1007/s41019-024-00246-x

[31] Battaglia, P.W., Hamrick, J.B., Bapst, V., Sanchez-Gonzalez, A., Zambaldi, V., Malinowski, M., Tacchetti, A., Raposo, D., Santoro, A., Faulkner, R., et al.: Relational inductive biases, deep learning, and graph networks. arXiv preprint arXiv:1806.01261 (2018)

[32] Liu, H., Wei, J., Xu, T.: Community detection based on community perspective and graph convolutional network. Expert Systems with Applications **231** (2023) https://doi.org/10.1016/j.eswa.2023.120748

[33] Liping Deng, B.G., Zheng, W.: GCN-based weakly-supervised community detection with updated structure centres selection. Connection Science **36**(1), 2291995 (2024) https://doi.org/10.1080/09540091.2023.2291995

[34] Amintoosi, M.: Overlapping clusters in cluster graph convolutional networks. Journal of Algorithms and Computation **53**(2), 33–45 (2021)

[35] Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W.L., Leskovec, J.: Graph convolutional neural networks for web-scale recommender systems. In: KDD (2018)

[36] Chen, Y., Wu, L.: In: Wu, L., Cui, P., Pei, J., Zhao, L. (eds.) Graph Neural Networks: Graph Structure Learning, pp. 297–321. Springer, Singapore (2022). https://doi.org/10.1007/978-981-16-6054-2_14 . https://doi.org/10.1007/978-981-16-6054-2_14

[37] Amintoosi, M.: Urban traffic prediction with learning based graph convolutional networks. In: Proceedings of the 55^{th} Annual Iranian Mathematics Conference, Ferdowsi University of Mashhad, pp. 145–148 (2024). (In Persian). https://github.com/mamintoosi/TGCN-GSL-TF

[38] Yu, B., Lee, Y., Sohn, K.: Forecasting road traffic speeds by considering area-wide spatio-temporal dependencies based on a graph convolutional neural network (gcn). Transportation Research Part C: Emerging Technologies **114**, 189–204 (2020) https://doi.org/10.1016/j.trc.2020.02.013